

OCL: Object Constraint Language — Uma Resenha

Luiz Arthur Costa E Costa

Resenha do artigo: Cabot, J. & Gogolla, M. — *Object Constraint Language (OCL): a Definitive Guide*

Contexto e motivação

O artigo de Jordi Cabot e Martin Gogolla oferece um panorama abrangente da Object Constraint Language (OCL), linguagem formal textual criada como complemento à UML (Unified Modeling Language). O objetivo central é suprir as limitações das notações gráficas, que por natureza não conseguem capturar todos os detalhes relevantes de um sistema. Linguagens gráficas como a UML priorizam a facilidade de uso em detrimento da expressividade: um diagrama de classes pode representar entidades, atributos e relacionamentos, mas não responde a perguntas do tipo 'pessoas na lista negra podem alugar carros?' ou 'como o preço de um aluguel é calculado?'. A OCL entra justamente para preencher essa lacuna, permitindo especificações precisas por meio de invariantes, regras de derivação, pré e pós-condições de operações e expressões de consulta.

Historicamente, a OCL surgiu em 1995 dentro da IBM como evolução de uma linguagem de expressões do método Syntropy, sendo integrada à UML em 1997. Desde então, expandiu consideravelmente seu escopo: de linguagem de restrição para diagramas UML, tornou-se um componente central da engenharia dirigida por modelos (MDE), sendo utilizada em transformações de modelos, definição de linguagens específicas de domínio (DSLs) e geração de código.

Conteúdo técnico

A parte central do artigo é densa e bem estruturada. Os autores descrevem o sistema de tipos da OCL com riqueza de detalhes: tipos atômicos predefinidos (Integer, Real, String, Boolean), tipos definidos pelo usuário (classes e enumerações), tipos parametrizados (Set, Bag, Sequence, OrderedSet) e tipos de tupla. Uma contribuição didática relevante é a distinção entre as quatro coleções concretas: Set é insensível à ordem e à frequência de inserção; Bag é insensível à ordem, mas sensível à frequência; Sequence é sensível a ambas; e OrderedSet é sensível à ordem, mas não à frequência. Essa distinção, ilustrada com tabelas de igualdade e predicados OCL formais, é um dos pontos mais esclarecedores do texto.

A seção sobre navegação em diagramas de objetos, operações de coleção (como *select*, *collect*, *forAll*, *exists* e *iterate*) e a lógica de três valores que emerge do valor *null* formam um material de referência valioso para quem deseja dominar a linguagem. O tratamento do operador *iterate* — apresentado como a operação mais poderosa, da qual as demais são casos especiais — é particularmente elucidativo, inclusive com exemplos paralelos em SQL que facilitam a compreensão para leitores familiarizados com bancos de dados relacionais.

Os diferentes tipos de expressão OCL são apresentados de forma progressiva: invariantes (a forma mais comum, que restringem o estado válido de instâncias de uma classe), expressões de inicialização, regras de derivação, operações de consulta e contratos de operação com pré e pós-condições. O estudo de caso da empresa de aluguel de automóveis EU-Rent percorre todo o artigo como fio condutor, ancorando os conceitos em um contexto concreto e

reconhecível.

Ferramentas e aplicações

O artigo cataloga o suporte ferramental disponível à época: parsers como MDT/OCL e Dresden OCL; ferramentas UML com suporte a OCL como ArgoUML, MagicDraw e Enterprise Architect; ambientes de validação e verificação como USE, UML2Alloy e UMLtoCSP; além de abordagens de geração de código. O levantamento é útil para contextualizar o ecossistema em torno da linguagem, embora inevitavelmente apresente limitações de atualidade dado o período de publicação. Os autores reconhecem que a maioria das ferramentas MDD não inclui geração de código a partir de expressões OCL, o que representa uma lacuna prática significativa.

A discussão sobre verificação formal é honesta quanto às limitações: o raciocínio sobre OCL é indecidível em geral, o que leva as ferramentas disponíveis a adotar uma de três estratégias — exigir interação do usuário (como o HOL-OCL, baseado no provador de teoremas Isabelle), restringir os construtores OCL utilizáveis, ou adotar verificação limitada por escopo, onde os domínios de busca são finitos por definição.

Agenda de pesquisa

Os autores apontam quatro desafios relevantes: (1) a necessidade de modularização e bibliotecas extensíveis para que usuários possam importar operadores especializados sem carregar toda a biblioteca padrão; (2) melhorias na sintaxe concreta e na semântica formal da linguagem, que ainda carece de uma definição completa; (3) técnicas de raciocínio eficiente sobre expressões OCL em modelos de grande escala, inclusive com uso de avaliação incremental e lazy; e (4) a consolidação de uma comunidade de praticantes que una pesquisadores e profissionais da indústria. Esses pontos demonstram visão de longo prazo e continuam pertinentes mesmo anos após a publicação.

Avaliação crítica

O texto é tecnicamente rigoroso e serve bem tanto como introdução quanto como referência. A estrutura é coerente: parte da motivação prática, avança para a descrição formal da linguagem, apresenta o ecossistema de ferramentas e conclui com uma agenda de pesquisa. A escolha de um caso de uso único e recorrente (EU-Rent) ao longo de todo o artigo é uma decisão pedagógica acertada, pois evita a fragmentação que costuma afetar textos de referência.

No entanto, alguns pontos merecem ressalvas. A profusão de exemplos com sintaxe de baixo nível — indispensável para uma apresentação rigorosa — pode intimidar leitores iniciantes sem formação em linguagens formais. A discussão sobre verificação formal, embora correta, toca na indecidibilidade do raciocínio sobre OCL sem aprofundar as implicações práticas para quem projeta sistemas reais. Além disso, a seção de ferramentas, como já mencionado, envelheceu, e alguns recursos citados já não estão mais disponíveis ou foram substituídos.

No conjunto, o artigo cumpre bem sua proposta de oferecer um guia abrangente sobre a OCL. É leitura recomendada para engenheiros de software interessados em modelagem precisa, para pesquisadores da área de MDE e para quem deseja compreender como linguagens formais podem complementar notações gráficas na especificação de sistemas complexos. A

clareza expositiva dos autores, aliada à profundidade técnica, torna o texto uma referência sólida nessa área.